

Relay: A collaborative UI model for design handoff

James Lin
Google
Mountain View, CA, USA
jameslin@google.com

Tiffany Kneareem
Google
Cambridge, MA, USA
tkneareem@google.com

Kris Giesing
Google
Mountain View, CA, USA
kgiesing@google.com

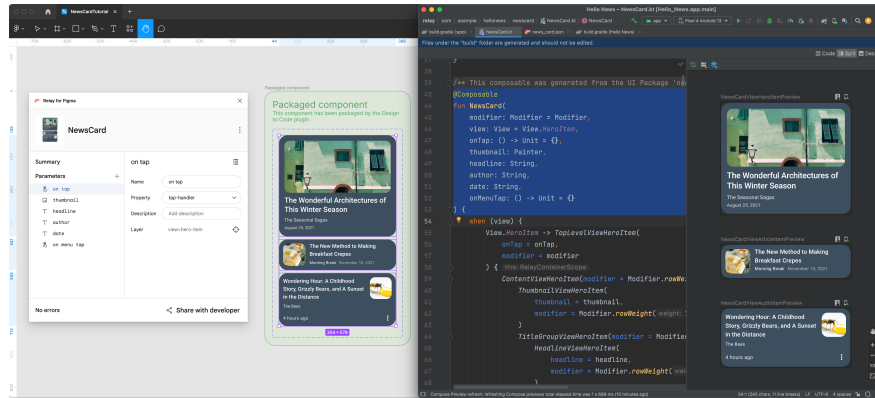


Figure 1: Left: The Relay for Figma plugin with a packaged component. Right: Jetpack Compose code generated by the Relay Gradle plugin, from the Figma component that was imported by the Relay for Android Studio plugin.

ABSTRACT

The design handoff process refers to the stage in the user interface (UI) design process where a designer gives their finished design to a developer for implementation. However, design decisions are lost when developers struggle to interpret and implement the designer's original intent. To address this problem, we built a system called Relay that utilizes concrete UI models to capture design intent. To our knowledge, Relay is the first system described in the academic literature to take artifacts from an existing design tool and generate a UI model.

CCS CONCEPTS

• **Human-centered computing** → **User interface programming**; • **Software and its engineering** → *Integrated and visual development environments*.

KEYWORDS

UI design, front-end development, design tools, UI model

ACM Reference Format:

James Lin, Tiffany Kneareem, and Kris Giesing. 2023. Relay: A collaborative UI model for design handoff. In *The 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23 Adjunct)*, October 29–November 01, 2023, San Francisco, CA, USA. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3586182.3616624>

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

UIST '23 Adjunct, October 29–November 01, 2023, San Francisco, CA, USA

© 2023 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0096-5/23/10.

<https://doi.org/10.1145/3586182.3616624>

1 INTRODUCTION

The term *design handoff* describes the process by which a designer hands over a finished user interface (UI) design to a software engineer for implementation through code. This is a crucial part of the software design process [4]. The design handoff becomes problematic in two key ways. The first is when designs are incomplete such as missing vital details (e.g., color and typographic styles), when the designer misses edge cases, or if the designer is unaware of technical limitations which prevent implementation. As such, design decisions are lost when developers struggle to interpret and implement the designer's original intent [6]. The second is when designers and developers disagree on whether the design should conform to the implementation or the original spec [3].

We created Relay to reduce friction in design handoff by using UI models [5] to encode design intent, from which Relay generates code. One issue with designers using UI models directly is that they are forced to think at a high level of abstraction too early in the design process, when they are accustomed to starting with concrete interfaces [7, 10]. Also, specifying models often resemble programming, with which most designers have limited experience. Specifying models thus impedes their main task of designing UIs. There have been attempts to simplify this, by presenting models as outlines [8] or other graphical notations (e.g., [2]). However, these efforts have not been incorporated into designers' existing tools.

In contrast, a designer using Relay does not create or interact with a UI model directly. Instead, the designer annotates their design with their design intent with Relay in Figma, and then Relay creates a UI model from the annotated Figma design when a developer imports it into Android Studio. To our knowledge, Relay is the first system described in the academic literature that lets designers create UI models from their existing design tools. Relay's UI models

are *concrete UI models*, where the model concretely defines the layout and interaction for a particular modality (e.g., desktop or mobile) independent of a platform (e.g., Windows or Android) [1].

2 USER INTERFACE

Relay is a set of plugins for tools that designers and developers already use. The overall workflow for Relay starts when the designer uses Relay within Figma to annotate their design with design intent. Their developer partner then uses Relay within Android Studio to import the Figma design, which converts the design into a UI model. When the developer compiles their code, Relay translates the UI model to Android code, faithfully rendering the design from Figma.

Designers use the **Relay for Figma plugin**, which is installed via the Figma Community website.¹ In Figma, a designer takes a portion of their UI design and converts it into a Figma *component*, which lets it be easily reused in other Figma designs. They open the Relay for Figma plugin and click on “Create UI Package”, which packages the component into a *UI package*. The designer can now add *parameters* to their component through the plugin. There are two types of parameters: *Content parameters* let the designer specify that a specific element, such as an image or a piece of text, should be dynamic data that is set by the application, such as the photo or title of a news article. *Interaction handlers* let the designer specify that a user should be able to interact with an element, such as tapping. The plugin uses the Figma Plugin API to store the UI package information as *plugin data* on the component’s root node.

For developers, there are two plugins. The **Relay for Android Studio plugin** is installed via the JetBrains Marketplace.² It allows developers to import and update UI packages from a Figma file. The **Relay Gradle plugin** is added to a project via a configuration file. It takes a UI package and generates Android UI code using Jetpack Compose, which is Android’s preferred UI framework [9].

In Android Studio, a software developer imports the UI package by opening the *Import UI Package* dialog, provided by the Relay for Android Studio plugin, and pastes the URL of the Figma file. The Relay for Android Studio plugin imports the UI package into Android Studio by calling the Figma REST API to retrieve the Figma file. Within the file, it finds the node corresponding to the UI package (which includes Relay for Figma plugin data). It then transforms the node into a concrete UI model. The UI model is saved, along with the design’s fonts, images, and other metadata, into the developer’s project.

When the developer compiles the project, the Relay Gradle plugin converts the UI package’s UI model to Jetpack Compose code, including a composable function (Compose’s term for a UI component) for each UI package. It includes a parameter for each parameter that the designer specified in Figma. The generated code also includes a preview of the composable function, where the developer can see that the generated code matches the Figma design, including all of the colors, corner radii, and font styles.

Relay also supports mapping Figma styles to Compose themes (so that the generated code uses named constants instead of hard-coded values) and mapping a UI package from Figma to an existing composable function (so that an app’s UI can use existing code).

3 IMPLEMENTATION OF UI PACKAGES

In an Android Studio project, a UI package is represented as a folder containing multiple files, including a concrete UI model file and related files such as fonts and images. The UI model file is a JSON file that includes a list of a component’s elements (such as text, images, groups, and other components), and then one or more variants that specify which elements are in a node and how they should be laid out. This lets the model support Figma’s variant feature (where each variant of a component can have a different layout), and it allows a component to be targeted to multiple contexts (such as desktop versus mobile, or light mode versus dark mode). The UI model also includes input parameters for the component, such as a text element’s contents or a button’s click handler. The model makes it possible for Relay to add support for more platforms in the future, such as the web.

In contrast, the Relay for Figma plugin implements a UI package as additional plugin data on a Figma component node. The plugin data includes the parameters and interaction handlers that the designer specified in the plugin UI. The data along with the Figma component sub-tree contains enough information to create a concrete UI model. The model is not stored explicitly, because Figma plugins are not intended to store large amounts of data within a Figma file, such as a separate concrete UI model.

4 USER FEEDBACK AND CONCLUSION

We conducted a multi-month field study with ten diverse companies who took part in our Early Access Program prior to the launch of this tool in late 2022. We reached out to users (four product designers and seven UI developers from six front-end UI teams) who volunteered to give us feedback. Overall, designers and developers were positive about the concept and thought Relay has potential to provide a better handoff experience than could be had with their current methods. Designers said that the Figma plugin fit their usual workflow and the benefits of incorporating Relay would outweigh the costs associated with learning how to use it. Although designers had to spend more time upfront packaging components into UI packages, once that effort was complete, the UI packages could be reused, speeding up future implementations with fewer back-and-forth iterations.

Developers thought that the quality of the generated code was high overall, “*I had no problem reading the generated code and understanding what it was doing*”. However, some developers ran into issues where they would need to change the code, e.g., naming parameters, since those names came from the designer in Figma and sometimes didn’t match the existing code. Any changes made to the generated code were overwritten if new version of the UI package was pulled in. There were other times when developers wanted to contribute to the UI package through modifying code, e.g., technical constraints or asking designers for unspecified behaviors. However, Relay does not currently support this.

As this paper has shown, embedding a UI model into an existing design process gives designers the benefits of using such models to specify design intent, without requiring them to learn technical skills to maintain a UI model directly. While the initial feedback suggests that is a promising direction, more work is needed to determine the boundaries of usefulness for designers and developers.

¹<https://www.figma.com/community>

²<https://plugins.jetbrains.com>

ACKNOWLEDGMENTS

We acknowledge the current and former members of the Relay team for their continuous work on this project, especially Damon Seeley, Justin Belcher, Travis Henspeter, Dian Xiang, and Emily Shack. We also thank Frank Bentley and Clara Kliman-Silver for their valuable feedback.

REFERENCES

- [1] Gaëlle Calvary, Joëlle Coutaz, David Thevenin, Quentin Limbourg, Laurent Bouillon, and Jean Vanderdonckt. 2003. A Unifying Reference Framework for multi-target user interfaces. *Interacting with Computers* 15, 3 (06 2003), 289–308. [https://doi.org/10.1016/S0953-5438\(03\)00010-9](https://doi.org/10.1016/S0953-5438(03)00010-9) arXiv:<https://academic.oup.com/iwc/article-pdf/15/3/289/7800242/iwc15-0289.pdf>
- [2] Pedro Campos and Nuno Nunes. 2007. Towards useful and usable interaction design tools: CanonSketch. *Interacting with Computers* 19, 5-6 (06 2007), 597–613. <https://doi.org/10.1016/j.intcom.2007.05.006> arXiv:<https://academic.oup.com/iwc/article-pdf/19/5-6/597/2548755/iwc19-0597.pdf>
- [3] K. J. Kevin Feng, Tony W Li, and Amy X. Zhang. 2023. Understanding Collaborative Practices and Tools of Professional UX Practitioners in Software Organizations. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) (CHI '23). Association for Computing Machinery, New York, NY, USA, Article 764, 20 pages. <https://doi.org/10.1145/3544548.3581273>
- [4] K.J. Kevin Feng and Amy X. Zhang. 2022. From Handoffs to Co-Creation: Deepening Collaboration between Designers, Developers, and Data Science Workers in UX Design. In *Proceedings of the InContext: Futuring User-Experience Design Tools Workshop at CHI Conference on Human Factors in Computing Systems* (CHI 2022). <https://hcibook.net/incontext/wp-content/uploads/sites/5/2022/04/From-Handoffs-to-Co-Creation-Deepening-Collaboration-between-Designers-Developers-and-Data-Science-Workers-in-UX-Design-1.pdf>
- [5] Model-Based User Interfaces Incubator Group. 2010. *Model-Based UI XG Final Report*. Retrieved January 7, 2023 from <https://www.w3.org/2005/Incubator/model-based-ui/XGR-mbui/>
- [6] Germán Leiva, Nolwenn Maudet, Wendy Mackay, and Michel Beaudouin-Lafon. 2019. Enact: Reducing designer–developer breakdowns when prototyping custom interactions. *ACM Transactions on Computer-Human Interaction (TOCHI)* 26, 3 (2019), 1–48.
- [7] Mark W. Newman and James A. Landay. 2000. Sitemaps, Storyboards, and Specifications: A Sketch of Web Site Design Practice. In *Proceedings of the 3rd Conference on Designing Interactive Systems: Processes, Practices, Methods, and Techniques* (New York City, New York, USA) (DIS '00). ACM, New York, NY, USA, 263–274. <https://doi.org/10.1145/347642.347758>
- [8] Angel Puerta, Michael Micheletti, and Alan Mak. 2005. The UI Pilot: A Model-Based Tool to Guide Early Interface Design. In *Proceedings of the 10th International Conference on Intelligent User Interfaces* (San Diego, California, USA) (IUI '05). ACM, New York, NY, USA, 215–222. <https://doi.org/10.1145/1040830.1040877>
- [9] Jolanda Verhoef. 2022. *Jetpack Compose 1.2 is now stable!* Retrieved January 7, 2023 from <https://android-developers.googleblog.com/2022/07/jetpack-compose-1-2-is-now-stable.html>
- [10] Annette Wagner. 1990. Prototyping: A Day in the Life of an Interface Designer. *The Art of Human-Computer Interface Design* (1990), 79–84.